STUFX - STUDENT STUDY TRACKER

STUDENT NAME - KARUMANCHI BHAVANA

STUDENT ID - 2520030253

CLUSTER - 01

SECTION - 03

Under the Guidance of
Supervisor Name -

ABSTRACT

- Smart Student Productivity Tracker is a web-based application designed to help students manage their study activities efficiently.
- HelpS students monitor learning progress and productivity levels.
- The system provides visual insights such as dashboards and statistics.
- Developed using modern web technologies to ensure accessibility and ease of use.
- Provides a platform to track daily study hours, manage subjects, and organize tasks.
- Enables students to maintain consistent study habits and better time management.
- The system provides visual insights such as dashboards and statistics.
- Useful for students preparing for exams, assignments, and long-term learnin goals.

EXISTING SYSTEM

- Students usually track study progress using manual methods such as notebooks or planners.
- Lack of a centralized system to store study data and task lists.
- Difficult to monitor daily or weekly study hours accurately.
- No automatic system to analyze productivity or generate reports.
- Students may forget pending tasks or study schedules.
- Manual tracking leads to poor organization and inefficient time management.
- Limited motivation due to lack of progress visualization.
- Existing methods do not support digital reminders or structured task management

PROPOSED SYSTEM

- A digital platform that allows students to monitor and manage their study activities
- Provides dashboard displaying study statistics and progress information.
- Allows students to add subjects, tasks, and track study sessions.
- Includes a study timer to record study duration accurately.
- Provides reports and charts for productivity analysis.
- Helps students maintain organized schedules and study routines.
- Encourages self-discipline and goal-oriented learning.
- Designed with a simple user interface for easy navigation and accessibility.

SYSTEM ARCHITECTURE

- The system follows a three-layer architecture for efficient processing and data management.

Presentation Layer (Frontend)

- Provides the user interface for interaction.
- Developed using HTML, CSS, JavaScript, and React.
- Displays dashboard, task lists, and study reports.

Application Layer (Backend)

- Handles business logic and application processing.
- Developed using Java programming language.
- Processes user requests and manages study data.

Data Layer (Storage)

- Stores student study records, tasks, and progress data.
- Data can be stored in database or local storage.

System Flow

- User → Frontend Interface → Backend Processing → Data Storage → Response to User

KLH (DEEMED TO BE UNIVERSITY)

Task HashMap (Search & Manage)

- Stores tasks using a HashMap for fast access.
- Enables quick searching, updating, and deletion of tasks
- Improves overall task management efficiency.

Searching and Sorting

- Implements search algorithms to find tasks quickly.
- Uses sorting techniques to arrange tasks by priority, date, or name.
- Provides an organized task view.

Task Data (ArrayList)

- All tasks are dynamically stored in an ArrayList.
- Allows easy addition, removal, and iteration of tasks.

File Storage

- Tasks are stored in a tasks.txt file.
- Ensures data persistence even after the application is closed.

Priority Queue

- Maintains tasks based on their priority levels.
  - Automatically arranges tasks so that higher priority tasks appear first

- **Conclusion (last line for slide):**
- The system uses multiple data structures like Stack, Queue, Heap, HashMap, and ArrayList to efficiently manage study tasks.

PROJECT MODULES

1. User Module

- User registration and login functionality
- Secure authentication using username and password

- **User profile management**
- Access to system features based on user role

2. Task / Data Management Module

- Create, update, and delete tasks or records
- Store user data and project information
- Organize data in structured format
- Provide easy retrieval of stored information

3. Database Module

- Stores all application data securely
- Maintains user details, tasks, and records
- Ensures data consistency and integrity
- Supports efficient data retrieval and updates

```java
import java.util.*;
import java.util.regex.*;

public class AuthSystem {

    static Scanner sc = new Scanner(System.in);

    static ArrayList<User> users = new ArrayList<>();

    static Pattern passwordPattern =
            Pattern.compile("^(?=.*[A-Z])(?=.*\\d)(?=.*[!@#$%^&*]).{8,}$");

    static User login(){

        while(true){

            System.out.println("=== Login / Sign up ===");

            System.out.print("Enter username: ");
            String uname = sc.nextLine();

            System.out.print("Enter password: ");
            String pass = sc.nextLine();

            for(User u: users){

                if(u.username.equals(uname)){

                    if(u.password.equals(pass)){

                        System.out.println("Login successful! Welcome "+uname);
                        return u;
```

```java
or(User var3 : users) {
  if (var3.username.equals(var0)) {
    if (var3.password.equals(var1)) {
      System.out.println("Login successful! Welcome " + var0);
      return var3;
    }

    System.out.println("Invalid password!");
  }


f (passwordPattern.matcher(var1).matches()) {
  User var4 = new User(var0, var1);
  users.add(var4);
  System.out.println("Account created! Welcome " + var0);
  return var4;


ystem.out.println("Password invalid!");



{
 new Scanner(System.in);
s = new ArrayList();
wordPattern = Pattern.compile("^(?=.*[A-Z])(?=.*\\d)(?=.*[!@#$%^&*]).{8,}$");
```

```java
import java.util.PriorityQueue;

public class TaskHeap {
   PriorityQueue<Task> heap = new PriorityQueue((var0, var1) -> var1.priority - var0.priority);

   public TaskHeap() {
   }

   void add(Task var1) {
      this.heap.add(var1);
   }

   Task poll() {
      return (Task)this.heap.poll();
   }

   void printHeap() {
      if (this.heap.isEmpty()) {
         System.out.println("No high priority tasks");
      } else {
         PriorityQueue var1 = new PriorityQueue(this.heap);

         while(!var1.isEmpty()) {
            Task var2 = (Task)var1.poll();
            System.out.println(var2.name + " (Priority:" + var2.priority + ") Deadline:" + var2.deadline);
         }
```

KLH (DEEMED TO BE UNIVERSITY)

4. Processing / Logic Module

- Handles core application logic
- Processes user inputs and performs operation
- Validates data before storing in database
- Executes algorithms or system functionalities

5. Interface (UI) Module

- Provides user-friendly interface
- Allows interaction between user and system
- Displays data and results clearly
- Ensures smooth navigation through application

6. Security Module

- Protects user data and system resources
- Implements authentication and authorization
- Prevents unauthorized access
- Maintains system privacy and safety

KLH (DEEMED TO BE UNIVERSITY)

OBJECTIVES

- To develop a system that helps students track study activities efficiently.
- To provide a digital platform for organizing study tasks and subjects
- To monitor daily, weekly, and monthly study hours.
- To improve student productivity through progress analysis.
- To help students build better time management habits.
- To provide visual insights into learning patterns and performance
- To encourage self-monitoring and disciplined study routines.

Course Outcome Attainment

- IMPLEMENTATION OF COURSE OUTCOMES OF DSA IN STUFX

CO1 – Searching Algorithms

Linear Search is implemented to find tasks in the task list during operations like searching, completing, and deleting tasks.

CO2 – Sorting Algorithms

Tasks are sorted based on deadlines using sorting techniques to organize and manage tasks efficiently.

CO3 – List ADT

ArrayList is used to store and manage study tasks dynamically, allowing easy insertion and traversal of tasks.

CO4 – Stack ADT

Stack is used to store completed tasks following the Last-In First-Out (LIFO) principle.

CO5 – Queue ADT

Queue is used to manage pending tasks following the First-In First-Out (FIFO) principle.

Screenshots for DSA


```java
// ===================== HASHING (search tasks) =====================
class TaskHash {
    HashMap<String, Task> map = new HashMap<>();
    void add(Task t){ map.put(t.name, t);}
    Task search(String name){ return map.getOrDefault(name,defaultValue: null);}
}
```

```java
// ===================== USER =====================
class User {
    String username;
    String password;
    User(String u, String p){ username=u; password=p; }
}
```

```java
// ===================== SEARCHING & SORTING =====================
class SearchingSorting {
    // Quick Sort by priority
    public static void quickSortByPriority(Task[] arr,int low,int high){
        if(low<high){
            int pi = partition(arr,low,high);
            quickSortByPriority(arr,low,pi-1);
            quickSortByPriority(arr,pi+1,high);
        }
    }
    private static int partition(Task[] arr,int low,int high){
        Task pivot = arr[high];
        int i=low-1;
        for(int j=low;j<high;j++){
            if(arr[j].priority > pivot.priority){
                i++;
                Task temp = arr[i]; arr[i]=arr[j]; arr[j]=temp;
            }
        }
        Task temp = arr[i+1]; arr[i+1]=arr[high]; arr[high]=temp;
        return i+1;
    }
}
```

```java
// ===================== STACK (recent completed) =====================
class TaskStack {
    LinkedList<Task> stack = new LinkedList<>();
    void push(Task t){ stack.addFirst(t);}
    Task pop(){ return stack.isEmpty()?null:stack.removeFirst();}
    void printStack(){
        if(stack.isEmpty()){ System.out.println(x: "No recent completed tasks"); return;}
        for(Task t: stack) System.out.println(t.name+" (Priority:"+t.priority+") Deadline:"+t.deadline);
    }
}
```

```java
// ===================== QUEUE (today tasks) =====================
class TaskQueue {
    LinkedList<Task> queue = new LinkedList<>();
    void enqueue(Task t){ queue.addLast(t);}
    Task dequeue(){ return queue.isEmpty()?null:queue.removeFirst();}
    void printQueue(){
        if(queue.isEmpty()){ System.out.println(x: "No tasks for today"); return;}
        for(Task t: queue) System.out.println(t.name+" (Priority:"+t.priority+") Deadline:"+t.deadline);
    }
}
```

```java
// ===================== MAIN PROJECT =====================
public class SmartStudyTrackerDSA {
    static Scanner sc = new Scanner(System.in);
    static ArrayList<User> users = new ArrayList<>();
    static Pattern passwordPattern = Pattern.compile(regex: "^(?=.*[A-Z])(?=.*\\d)(?=.*[!@#$%^&*])[A-Za-z\\d!@#$%^&*]{8,}$");

    // Login / Signup
    static User login(){
        while(true){
            System.out.println(x: "=== Login / Sign Up ===");
            System.out.print(s: "Enter username: "); String uname = sc.nextLine();
            System.out.print(s: "Enter password: "); String pass = sc.nextLine();

            // Check existing
            for(User u: users){
                if(u.username.equals(uname)){
                    if(u.password.equals(pass)){
                        System.out.println("Login successful! Welcome "+uname+"\n");
                        return u;
                    } else{
                        System.out.println(x: "Invalid password! Try again.\n");
                        continue;
                    }
                }
            }
```

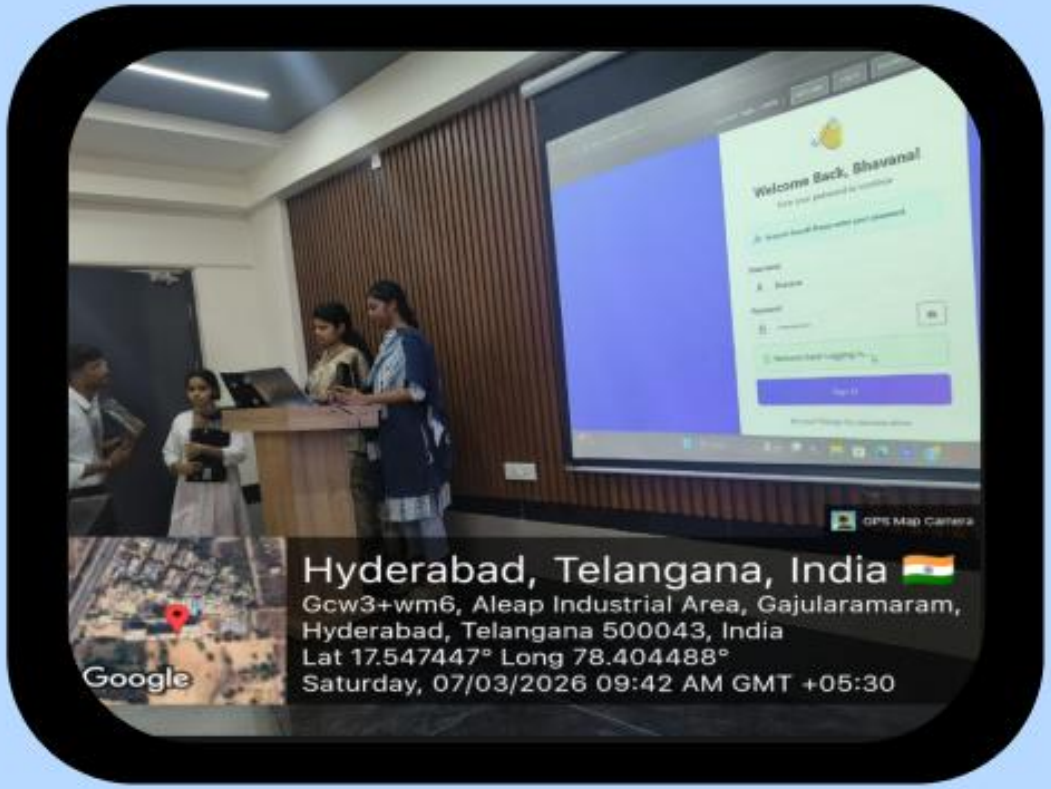
```java
// Dashboard
static void dashboard(){
    TaskStack completedStack = new TaskStack();
    TaskQueue todayQueue = new TaskQueue();
    TaskHeap priorityHeap = new TaskHeap();
    TaskHash taskMap = new TaskHash();
    ArrayList<Task> allTasks = new ArrayList<>();

    while(true){
        System.out.println(x: "\n=== Dashboard ===");
        System.out.println(x: "1. Add Task");
        System.out.println(x: "2. View Today's Tasks");
        System.out.println(x: "3. Mark Task Completed");
        System.out.println(x: "4. View Recent Completed Tasks");
        System.out.println(x: "5. View High Priority Tasks");
        System.out.println(x: "6. Sort Tasks by Priority");
        System.out.println(x: "7. Search Task by Name");
        System.out.println(x: "8. Exit");
        System.out.print(s: "Choice: "); int choice = sc.nextInt(); sc.nextLine();
```

```java
    // New account
    if(!passwordPattern.matcher(pass).matches()){
        System.out.println(x: "Password invalid! Must be 8+ chars, 1 uppercase, 1 number, 1 special char.\n");
        continue;
    }

    User newUser = new User(uname, pass);
    users.add(newUser);
    System.out.println("Account created successfully! Welcome "+uname+"\n");
    return newUser;
}
```

```java
switch(choice){
    case 1:
        System.out.print(s: "Task Name: "); String n=sc.nextLine();
        System.out.print(s: "Priority (1-10): "); int p=sc.nextInt(); sc.nextLine();
        System.out.print(s: "Deadline (YYYY-MM-DD): "); String d=sc.nextLine();
        Task t = new Task(n,p,d);
        allTasks.add(t); todayQueue.enqueue(t); priorityHeap.add(t); taskMap.add(t);
        System.out.println(x: "Task added!");
        break;
    case 2:
        System.out.println(x: "=== Today's Tasks ===");
        todayQueue.printQueue();
        break;
    case 3:
        System.out.print(s: "Task Name to mark completed: "); String cname = sc.nextLine();
        Task ct = taskMap.search(cname);
        if(ct!=null && !ct.completed){
            ct.completed = true; completedStack.push(ct);
            System.out.println(x: "Task marked completed!");
        } else System.out.println(x: "Task not found or already completed!");
        break;
    case 4:
        System.out.println(x: "=== Recent Completed Tasks ===");
        completedStack.printStack();
        break;
    case 5:
        System.out.println(x: "=== High Priority Tasks ===");
        priorityHeap.printHeap();
        break;
```

```java
        break;
    case 6:
        Task[] arr = allTasks.toArray(new Task[0]);
        SearchingSorting.quickSortByPriority(arr,low: 0,arr.length-1);
        System.out.println(x: "=== Tasks Sorted by Priority ===");
        for(Task tt: arr){
            System.out.println(tt.name+" (Priority:"+tt.priority+") Deadline:"+tt.deadline+" Completed:"+tt.completed);
        }
        break;
    case 7:
        System.out.print(s: "Task Name to Search: "); String sname=sc.nextLine();
        Task st = taskMap.search(sname);
        if(st!=null) System.out.println("Found Task: "+st.name+" (Priority:"+st.priority+") Completed:"+st.completed+" Deadline:"+st.deadline);
        else System.out.println(x: "Task not found!");
        break;
    case 8:
        System.out.println(x: "Exiting dashboard. Bye!");
        return;
    default:
        System.out.println(x: "Invalid choice!");
    }
  }
}

Run | Debug
public static void main(String[] args){
    System.out.println(x: "Welcome to Smart Student Study Tracker (DSA Console Version)!");
    User u = login();
    dashboard();
  }
}
```

Geo- Tag Photos



THANK YOU...